Module 03: Routing & Controllers

Duration: 4-5 days | **Level:** Beginner-Intermediate

Defining Routes

Basic Routes (`routes/web.php`)

```
Route::get('/dashboard', [DashboardController::class,
'index'])->name('dashboard');

Route::middleware('auth')->group(function () {
Route::resource('projects', ProjectController::class);
Route::resource('tasks', TaskController::class);
Route::patch('/tasks/{task}/complete', [TaskController::class, 'complete'])
->name('tasks.complete');
});
```

Route Parameters

```
Route::get('/tasks/{task}', [TaskController::class, 'show']);
// {task} auto-resolves to Task model via Route Model Binding
```

Named Routes

Always name routes - use `route('tasks.show', $task)` in Blade instead of hardcoded URLs.

Resource Controllers

```
php artisan make:controller ProjectController --resource
```

Method	URI	Action	Route Name
GET	/projects	index	projects.index
GET	/projects/create	create	projects.create
POST	/projects	store	projects.store
GET	/projects/{project}	show	projects.show
GET	/projects/{project}/edit	edit	projects.edit
PUT/PATCH	/projects/{project}	update	projects.update
DELETE	/projects/{project}	destroy	projects.destroy

Controllers in TaskForge

Study these files:

- `app/Http/Controllers/ProjectController.php` - full CRUD with authorization
- `app/Http/Controllers/TaskController.php` - nested actions (comments, attachments, complete)
- `app/Http/Controllers/AuthController.php` - session auth

Controller Best Practices

1. **Thin controllers** - delegate complex logic to services (`DashboardService`)
2. **Type-hint dependencies** in constructor

3. **Use Form Requests** for validation (not inline in controller)
4. **Return explicit types**: `View`, `RedirectResponse`, `JsonResponse`

Route Model Binding

Laravel automatically injects models:

```
public function show(Task $task): View
{
// $task is already loaded from DB by ID in URL
$this->authorize('view', $task);
return view('tasks.show', compact('task'));
}
```

Custom binding in `AppServiceProvider`:

```
Route::bind('task', fn ($value) => Task::where('slug', $value)->firstOrFail());
```

API Routes (`routes/api.php`)

```
Route::middleware('auth:sanctum')->group(function () {
Route::apiResource('tasks', TaskApiController::class)->names('api.tasks');
});
```

API routes are prefixed with `/api` and use token auth (Sanctum).

Exercises

1. Add a route `GET /projects/{project}/archive` that sets `is_archived = true`.
2. Create `ProjectController@archive` with proper authorization.
3. List all TaskForge routes: `php artisan route:list --name=tasks`.

Next Module

-> Module 04: Blade Templates